

Курс Hugging Face «AI Agents Course»

Подробный конспект: Разделы 1-4 + все бонусные модули (курс целиком)

От основ агентов до финального проекта на бенчмарке GAIA и агентов в играх

Статус: Раздел 1 пройден полностью, итоговый тест сдан на 90%, сертификат «Certificate of Achievement — Fundamentals of Agents» получен и скачан. Разделы 2-4 и все три бонусных модуля изучены в теории и на примерах кода (фреймворки smolagents/LlamaIndex/LangGraph, сквозной пример Agentic RAG, структура финального проекта на бенчмарке GAIA, дообучение под function-calling, наблюдаемость агентов, агенты в играх). Ниже — полный конспект всего материала курса.

Модуль 0. Ввод в должность

Прежде чем начать изучать агентов, курс просит выполнить несколько организационных шагов. Аккаунт Hugging Face для прохождения курса уже создан и подключён (в этом конспекте — аккаунт KonstantinVCH), так что первый и самый важный пункт уже закрыт.

- Создать аккаунт Hugging Face — уже сделано.
- Присоединиться к серверу Discord курса и представиться в канале #introduce-yourself — по желанию, для общения с другими студентами.
- Подписаться на организацию курса «Hugging Face Agents Course» на Hub, чтобы не пропускать обновления.
- Поставить ★ репозиторию курса на GitHub и поделиться прогрессом в соцсетях — необязательно.

Если закончится бесплатный лимит на инференс: работа с моделью локально через Ollama

Курс использует облачный Hugging Face Inference API, у которого есть лимиты. Если лимит исчерпан, можно временно перейти на локальный запуск модели через Ollama:

```
# 1. Установить Ollama (см. официальную инструкцию ollama.com)
# 2. Скачать модель локально
ollama pull qwen2:7b
# 3. Запустить сервер Ollama в отдельном терминале
ollama serve
# 4. Установить поддержку LiteLLM для smolagents
pip install 'smolagents[litellm]'
```

А в коде агента вместо облачной модели подключить локальную через LiteLLMModel:

```
from smolagents import LiteLLMModel

model = LiteLLMModel(
    model_id="ollama_chat/qwen2:7b",          # или другая модель из Ollama
    api_base="http://127.0.0.1:11434",        # локальный сервер Ollama
    num_ctx=8192,
)
```

Идея простая: Ollama поднимает локальный сервер с API, совместимым с форматом OpenAI, а LiteLLMModel умеет говорить с любым провайдером в этом формате — поэтому замена облачной модели на локальную не требует переписывать остальной код агента.

Раздел 1. Введение в работу с агентами

Это самый объёмный и важный раздел курса: именно за его освоение и успешную сдачу итогового теста выдаётся сертификат «Fundamentals of Agents». Раздел закладывает фундамент: что такое агент, какую роль в нём играет языковая модель (LLM), как агент пользуется инструментами и как выглядит его рабочий цикл «Мышление → Действие → Наблюдение». В конце раздела предлагается собрать своего первого агента на библиотеке smolagents и опубликовать его в Hugging Face Spaces.

Что такое ИИ-агент

Разработчики курса объясняют идею агента на примере дворецкого по имени Алфред. Когда ему говорят «Алфред, хочу кофе», он не действует вслепую: сначала рассуждает и строит план (дойти до кухни, включить кофемашину, сварить кофе, принести), а затем выполняет план, используя доступные ему средства — в данном случае кофемашину. Ровно это и есть агент: система, которая понимает естественный язык, рассуждает и планирует, а затем действует в своей среде, чтобы добиться цели.

Более формально: агент — это система, которая использует AI-модель (обычно LLM) для взаимодействия со своей средой ради достижения цели, поставленной пользователем. Она сочетает в себе рассуждение, планирование и выполнение действий — зачастую через внешние инструменты.



Удобно представлять агента как два взаимодополняющих компонента. «Мозг» — это AI-модель, которая занимается рассуждением и планированием и решает, какое действие предпринять в конкретной ситуации. «Тело» — это набор инструментов и возможностей, которыми агент реально может воспользоваться: то, что он умеет физически или программно сделать. Диапазон доступных действий определяется именно телом — так же, как человек, не имея крыльев, не может «полететь», но может «пойти», «взять», «нажать».

Насколько агент «агентен»: спектр агентности

Не все системы, использующие LLM, одинаково самостоятельны. Библиотека smolagents предлагает удобную шкалу — от систем, где вывод модели вообще не влияет на ход программы, до полноценных многошаговых агентов и целых мультиагентных систем:

Уровень	Что делает вывод LLM	Как называется	Пример кода
☆☆☆	Никак не влияет на ход программы	Simple processor	<code>process_llm_output(llm_response)</code>
★☆☆	Определяет базовое ветвление	Router	<code>if llm_decision(): path_a() else: path_b()</code>
★★☆	Определяет, какая функция вызывается	Tool caller	<code>run_function(llm_chosen_tool, llm_chosen_args)</code>
★★★	Управляет циклом и продолжением работы	Multi-step Agent	<code>while llm_should_continue(): execute_next_step()</code>
★★★	Один агент запускает другого	Multi-Agent	<code>if llm_trigger(): execute_agent()</code>

«Мозгом» агента чаще всего служит LLM — модель, которая принимает текст и выдаёт текст (примеры: GPT-4, Llama, Gemini). Реже используются VLM (Vision Language Model) — модели, которые вдобавок понимают изображения. Сама по себе LLM умеет только генерировать текст: чтобы агент мог, скажем, нарисовать картинку или отправить письмо, разработчики добавляют ему дополнительные функции — Tools (инструменты), — которые модель умеет вызывать через специально сгенерированный текст.

Агентов такого рода уже используют в реальных продуктах: голосовые ассистенты вроде Siri и Alexa анализируют запрос, обращаются к нужным сервисам и выполняют действия (ставят напоминание, отправляют сообщение); чат-боты поддержки клиентов ведут диалог, ищут информацию во внутренних базах и заводят обращения; NPC в видеоиграх на основе LLM реагируют на игрока не по жёсткому сценарию, а контекстно, что делает их поведение более живым.

Языковые модели (LLM): «мозг» агента

LLM (Large Language Model) — это модель, обученная на огромных массивах текста, которая хорошо понимает и генерирует человеческий язык. В основе почти всех современных LLM лежит архитектура Transformer, построенная на механизме внимания (Attention); её популярность резко выросла после выхода модели BERT в 2018 году.

У трансформеров есть три основных варианта. Encoder-модели (например, BERT) превращают текст в компактное числовое представление — эмбединг; они хороши для классификации текста, семантического поиска и распознавания именованных сущностей. Decoder-модели (например, Llama) генерируют текст токен за токеном и лежат в основе большинства современных чат-моделей и генераторов кода. Seq2Seq-модели (например, T5, BART) сочетают энкодер и декодер и хорошо подходят для перевода и суммаризации. Подавляющее большинство современных LLM с миллиардами параметров — decoder-модели: GPT-4 (OpenAI), Llama 3 (Meta), Gemini (Google), Deepseek-R1 (DeepSeek), SmolLM2 (Hugging Face), Mistral.

Принцип работы у всех этих моделей один и тот же: предсказать следующий токен, опираясь на всю предыдущую последовательность. Токен — это не обязательно целое слово, а скорее «кусочек» слова: например, «interesting» может собираться из токенов «interest» и «ing». Благодаря этому словарь модели остаётся сравнительно небольшим (для Llama 2 — около 32 000 токенов) даже притом, что живых слов в языке гораздо больше.

Генерация продолжается до тех пор, пока модель не выдаст специальный токен конца последовательности — EOS (End Of Sequence). У каждой модели свой набор служебных токенов и свой EOS:

Модель	Провайдер	EOS-токен	Что означает
GPT-4	OpenAI	< endoftext >	конец текста сообщения
Llama 3	Meta	< eot_id >	конец последовательности
Deepseek-R1	DeepSeek	< end_of_sentence >	конец текста сообщения
SmolLM2	Hugging Face	< im_end >	конец сообщения/инструкции
Gemma	Google	<end_of_turn>	конец хода в диалоге

Раз модель предсказывает следующий токен, глядя на всю предыдущую последовательность (свой «промпт»), то формулировка запроса напрямую определяет качество ответа — отсюда и важность грамотного prompt-инжиниринга. Ещё одно ключевое понятие — context length: максимальное число токенов, которое модель способна удерживать во «внимании» одновременно.

Обучение LLM проходит в два этапа. Сначала — предобучение (pre-training) на огромном корпусе текста в режиме самообучения (модель просто учится предсказывать следующее слово), благодаря чему она усваивает структуру языка и общие знания о мире. Затем — дообучение (fine-tuning) под конкретную задачу: диалог, вызов инструментов, классификацию, генерацию кода. Использовать готовую LLM можно двумя способами — запустить локально (если хватает железа) или обратиться к ней через облачный API; в этом курсе используется в основном второй вариант, через Hugging Face Inference API.

Сообщения и специальные токены: как агент «помнит» диалог

В чат-интерфейсах вроде ChatGPT или HuggingChat создаётся ощущение, что модель «помнит» переписку. На самом деле это иллюзия, создаваемая интерфейсом: перед каждым обращением к модели вся история сообщений заново склеивается в один длинный текстовый промпт, и модель читает его целиком, с нуля, каждый раз. Мост между удобным списком сообщений (роли user/assistant/system) и этим единым текстовым промптом называется chat template — шаблон чата, специфичный для каждой модели.

Системное сообщение (System Message, или System Prompt) задаёт правила игры для всего диалога: например, «ты вежливый оператор поддержки» или, для контраста, «ты дерзкий бунтарь, не слушающийся пользователя» — и модель ведёт себя соответствующим образом на протяжении всей беседы. Для агента системное сообщение особенно важно: именно там перечисляются доступные инструменты, формат, в котором нужно описывать действия, и структура «мыслительного» процесса.

Один и тот же диалог у разных моделей превращается в совершенно разный по виду текстовый промпт. Например, для SmolLM2 диалог оформляется так:

```
<|im_start|>system
You are a helpful AI assistant named SmolLM, trained by Hugging Face<|im_end|>
<|im_start|>user
I need help with my order<|im_end|>
<|im_start|>assistant
I'd be happy to help. Could you provide your order number?<|im_end|>
<|im_start|>user
It's ORDER-123<|im_end|>
<|im_start|>assistant
```

А для Llama 3.2 — иначе, с другими служебными токенами (<|begin_of_text|>, <|start_header_id|>, <|eot_id|> и т.д.). Именно поэтому нельзя просто скопировать промпт, написанный под одну модель, и ожидать, что он корректно сработает на другой.

Ещё одно важное разграничение — Base Model и Instruct Model. Базовая модель обучена лишь предсказывать продолжение произвольного текста; инструктивная модель дополнительно дообучена следовать инструкциям и вести диалог (например, SmolLM2-135M — база, а SmolLM2-135M-Instruct — её инструктивная версия). Чтобы базовая модель вела себя как инструктивная, нужно правильно оформить промпт по формату chat template — один из распространённых стандартов такого формата называется ChatML.

На практике в библиотеке transformers всё это скрыто за одной функцией. Достаточно правильно оформить список сообщений и вызвать `apply_chat_template` — токенизатор сам подставит нужные служебные токены под конкретную модель:

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("HuggingFaceTB/SmolLM2-1.7B-Instruct")
rendered_prompt = tokenizer.apply_chat_template(
    messages, tokenize=False, add_generation_prompt=True
)
```

Инструменты (Tools): как агент выходит за рамки текста

LLM сама по себе умеет только одно — генерировать текст. Чтобы агент мог что-то реально сделать (посчитать, найти актуальную информацию в интернете, отправить письмо, нарисовать картинку), ему дают Tools — функции с чётко описанной целью. Типичные примеры: веб-поиск, генерация изображений, извлечение информации из внешнего источника (retrieval), обращение к внешнему API (GitHub, YouTube, Spotify и т.п.).

Хороший инструмент дополняет модель именно там, где у неё есть слабое место. Классический пример — калькулятор: он даст куда более надёжный результат, чем попытка модели «в уме» посчитать сложное выражение. Ещё более важный случай — актуальность данных: знания LLM ограничены датой обучения, и без инструмента вопрос про сегодняшнюю погоду приведёт не к ответу, а к правдоподобной галлюцинации.

Технически ни одна LLM не может «нажать кнопку» самостоятельно. Вместо этого агенту заранее, через системный промпт, описывают, какие инструменты существуют и как их вызывать. Когда модели нужно воспользоваться инструментом, она не выполняет его сама, а генерирует специальный текст — своего рода заявку на вызов, например `call weather_tool('Paris')`. Именно агент (программа вокруг модели)

читает эту заявку, реально выполняет вызов и возвращает результат обратно модели для дальнейшей работы. Пользователь всей этой кухни не видит — со стороны кажется, будто модель «сама» узнала погоду.

Чтобы модель поняла, как пользоваться инструментом, описание должно быть точным: как называется инструмент, что он делает, какие у него входы и каким будет выход. Вручную это можно оформить одной строкой:

```
Tool Name: calculator, Description: Multiply two integers., Arguments: a: int, b: int, Outputs: int
```

Писать такие описания руками неудобно и легко ошибиться, поэтому на практике их извлекают автоматически прямо из кода — из типов аргументов, docstring-a и имени функции — с помощью простого декоратора:

```
@tool
def calculator(a: int, b: int) -> int:
    """Multiply two integers."""
    return a * b

print(calculator.to_string())
# Tool Name: calculator, Description: Multiply two integers., Arguments: a: int, b: int, Outputs: int
```

Такой же принцип лежит в основе стандарта Model Context Protocol (MCP) — открытого протокола, который унифицирует то, как приложения предоставляют инструменты для LLM. MCP даёт готовый набор интеграций, позволяет безболезненно менять провайдера модели и задаёт общие правила безопасности данных — иначе говоря, избавляет разработчиков от необходимости заново изобретать интерфейс инструментов под каждый фреймворк.

Рабочий цикл агента: Мышление → Действие → Наблюдение

Вся предыдущая теория — про LLM, сообщения и инструменты — нужна для того, чтобы понять главный механизм, на котором держится любой агент: непрерывный цикл из трёх шагов, который продолжается до тех пор, пока цель не будет достигнута (по сути, обычный цикл while из программирования).



На шаге Мышление (Thought) языковая часть агента решает, что делать дальше — это внутреннее рассуждение, «внутренний монолог» модели. На шаге Действие (Action) агент вызывает конкретный инструмент с конкретными аргументами. На шаге Наблюдение (Observation) агент получает результат работы инструмента и анализирует его, чтобы решить, что делать на следующем витке цикла.

Разработчики курса иллюстрируют это на сквозном примере агента погоды по имени Алфред. Пользователь спрашивает: «Какая сейчас погода в Нью-Йорке?». Алфред рассуждает: нужно вызвать инструмент погоды. Он формирует действие в строгом формате:

```
Thought: I need to check the current weather for New York.
Action:
{
  "action": "get_weather",
  "action_input": {"location": "New York"}
}
```

Инструмент возвращает наблюдение — «переменная облачность, 15°C, влажность 60%» — и Алфред, уже располагая этими данными, формулирует финальный ответ пользователю. Если бы наблюдение показало ошибку или неполные данные, агент просто прошёл бы ещё один виток цикла, скорректировав подход.

Мышление и подход ReAct

«Мысль» — это внутреннее рассуждение и планирование агента: разбить задачу на шаги, проанализировать ошибку, выбрать вариант с учётом ограничений, вспомнить предпочтения пользователя из более раннего контекста, признать, что предыдущий подход не сработал, и попробовать иначе. У моделей, специально дообученных на вызов функций, этот шаг иногда можно пропустить.

Отдельная техника промптинга — Chain-of-Thought (CoT, «цепочка рассуждений»): модели предлагают «думать шаг за шагом», прежде чем дать окончательный ответ. Это хорошо работает для логических и математических задач и не требует никаких внешних инструментов:

Question: What is 15% of 200?
Thought: Let's think step by step. 10% of 200 is 20, and 5% of 200 is 10, so 15% is 30.
Answer: 30

Подход ReAct (Reasoning + Acting — «рассуждение + действие») идёт дальше: он чередует мысль, действие (вызов инструмента) и наблюдение (результат инструмента), что и позволяет решать составные, многошаговые задачи, требующие внешних данных:

Thought: I need to find the latest weather in Paris.
Action: Search["weather in Paris"]
Observation: It's 18°C and cloudy.
Thought: Now that I know the weather...
Action: Finish["It's 18°C and cloudy in Paris."]

Критерий	Chain-of-Thought	ReAct
Пошаговая логика	да	да
Внешние инструменты	нет	да (Actions + Observations)
Лучше всего подходит для	логики, математики, внутренних рассуждений	поиска информации, динамических многошаговых задач

Отдельно стоит упомянуть современные модели вроде Deepseek R1 или OpenAI o1: они обучены «думать» ещё на уровне тренировки, отделяя фазу рассуждения от финального ответа специальными токенами <think>...</think>. Это принципиально другой, более глубокий уровень, чем просто техника промптинга вроде ReAct или CoT.

Действие: как агент фактически что-то делает

Действие — это конкретный шаг, которым агент взаимодействует со средой: поиск в интернете, обращение к базе данных, управление устройством. По способу описания действия различают несколько типов агентов: JSON-агент описывает действие структурой JSON; агент кода (Code Agent) пишет исполняемый программный код, который затем интерпретируется извне; агент вызова функций (Function-calling Agent) — это разновидность JSON-агента, отдельно дообученная генерировать новое сообщение под каждое действие.

Поскольку LLM оперирует только текстом, критически важно, чтобы она вовремя остановилась — сразу после того, как полностью описала своё действие, не продолжая генерацию дальше. Это и есть подход «остановись и разбери» (stop and parse): модель генерирует действие в строго заданном формате, останавливает генерацию, а внешний парсер уже читает этот текст, определяет нужный инструмент и извлекает параметры для вызова.

Альтернатива JSON-формату — Code Agents, которые вместо структуры данных генерируют исполняемый код (чаще всего на Python). У этого подхода несколько практических преимуществ: код естественно выражает сложную логику (циклы, условия, вложенные вызовы), его проще переиспользовать и отлаживать, он напрямую интегрируется с внешними библиотеками. Обратная сторона — риск

безопасности при выполнении сгенерированного кода, поэтому на практике используют фреймворки со встроенными защитными механизмами, например smolagents.

Наблюдение: как агент учится на обратной связи

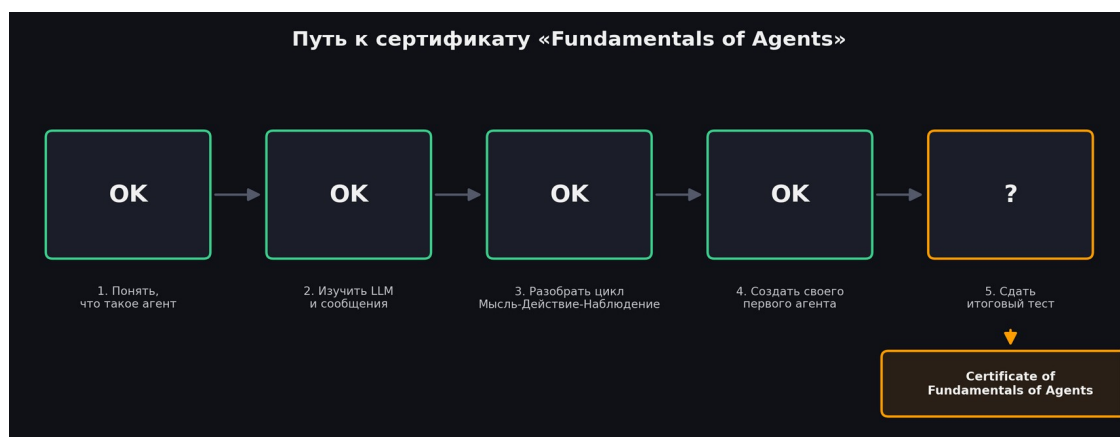
Наблюдение — это то, как агент воспринимает последствия собственных действий: сигналы из среды в виде данных API, сообщений об ошибках или системных логов, которые определяют следующий виток мышления. На этом шаге агент собирает обратную связь об успехе или неудаче своего действия, добавляет результат в свой контекст (по сути, обновляет память в рамках текущего диалога) и, опираясь на новую информацию, корректирует дальнейшую стратегию.

После каждого действия фреймворк агента выполняет три шага по порядку: разбирает действие, чтобы понять, какую функцию и с какими аргументами нужно вызвать; фактически выполняет это действие; и добавляет результат в контекст как наблюдение — именно оно станет частью следующего промпта для модели.

Практика: от «игрушечного» агента до smolagents

Раздел завершается двумя практическими ноутбуками в Google Colab. Первый — «Библиотека фиктивных агентов» (Dummy Agent Library): в нём вручную, без готового фреймворка, собирается системный промпт с описанием инструментов и циклом Thought-Action-Observation, чтобы своими руками увидеть, что на самом деле происходит «под капотом» у агентных библиотек, используя обычный Hugging Face Inference API. Второй ноутбук — уже настоящий агент на библиотеке smolagents: там определяются инструменты через декоратор `@tool`, подключается модель (через `InferenceClientModel` или локально через Ollama), и из этого собирается `CodeAgent`, решающий конкретную небольшую задачу. Ноутбук заканчивается публикацией агента в Hugging Face Spaces, чтобы им могли пользоваться и другие.

Путь к сертификату



Когда весь материал раздела пройден, остаётся последний шаг — Unit 1 Final Quiz. Тест встроен как отдельное пространство (Space) на Hugging Face Hub: [agents-course/unit_1_quiz](https://huggingface.co/spaces/agents-course/unit_1_quiz). Чтобы результат сохранился и по нему выдался сертификат, нужно войти через кнопку «Sign in with Hugging Face» (это стандартный OAuth-вход, запрашивающий только доступ к публичному профилю), пройти все вопросы по очереди кнопкой «Next», а в самом конце обязательно нажать «Submit» — без этого результат не попадёт на Hugging Face Hub, и сертификат не откроется.

Результат: тест пройден с результатом 90% (9 из 10 верных ответов; один вопрос был случайно пропущен из-за преждевременного клика «Next» до загрузки вариантов). Сертификат «Certificate of Achievement — Fundamentals of Agents» получен на имя Chaplygin Konstantin, дата — 20.08.2026, выдан за успешное прохождение Unit 1: Foundations of Agents в Hugging Face Agents Course. Сертификат скачан из интерфейса теста и доступен для публикации в LinkedIn через встроенную кнопку «Add to Profile».

Итог по Разделу 1

Раздел 1 полностью пройден: изучены понятия агента, LLM, сообщений и chat-шаблонов, инструментов и цикла Мышление-Действие-Наблюдение, а итоговый тест сдан с результатом 90%. Сертификат «Fundamentals of Agents» получен и сохранён.

Дальше курс продолжается тремя более практическими блоками: Раздел 2 разбирает три конкретных фреймворка для создания агентов (smolagents, LlamaIndex, LangGraph), Раздел 3 показывает сквозной пример Agentic RAG на основе агента-дворецкого Алфреда, а Раздел 4 — это финальный проект с самостоятельным созданием, тестированием и попыткой сертификации собственного агента на бенчмарке GAIA, за что выдаётся второй, более полный сертификат «Certificate of Completion». Ниже приведён конспект и этих разделов.

Раздел 2. Фреймворки для ИИ-агентов

Второй раздел курса переходит от теории к практике и знакомит с тремя популярными библиотеками для построения агентов: smolagents (лёгкий фреймворк от Hugging Face, где агент пишет и исполняет код), LlamaIndex (изначально библиотека для поиска по данным — RAG, — которая доросла до полноценного инструмента построения агентов) и LangGraph (фреймворк от LangChain, ориентированный на предсказуемость и контроль исполнения через граф состояний). Каждый инструмент решает одну и ту же задачу — соединить LLM, инструменты и цикл принятия решений, — но с разным балансом свободы и контроля.

2.1. Фреймворк smolagents

smolagents — это минималистичная библиотека Hugging Face для создания агентов. Её ключевая особенность и главное отличие от многих других фреймворков в том, что агент не описывает свои действия в формате JSON, а пишет и исполняет настоящий код на Python — отсюда и название подхода Code Agents.

Зачем нужен именно код, а не JSON

У код-агентов есть несколько практических преимуществ перед JSON-агентами. Композируемость: код естественно позволяет комбинировать результаты нескольких вызовов инструментов, использовать циклы и условия — то, что в JSON пришлось бы неуклюже размазывать по нескольким шагам. Управление объектами: код может напрямую работать со сложными структурами данных (списками, словарями, объектами), а не только с примитивами JSON. Универсальность: на языке программирования можно выразить практически любую вычислительную задачу. Естественность для LLM: современные модели уже рассмотрели огромное количество качественного программного кода при обучении, поэтому генерация кода часто получается у них более естественной и надёжной, чем генерация структурированного JSON.

CodeAgent и ToolCallingAgent

В smolagents есть два основных типа агентов. CodeAgent на каждом шаге пишет исполняемый Python-код, который сам вызывает нужные инструменты и обрабатывает их результат прямо внутри кода — это самый гибкий вариант, подходящий для сложной многошаговой логики. ToolCallingAgent, напротив, работает ближе к классическому JSON-подходу: он описывает вызов инструмента структурированным образом (как это делают модели со встроенным function calling), что даёт больше предсказуемости и проще для отладки, но менее гибко для сложных сценариев.

Критерий	CodeAgent	ToolCallingAgent
Формат действия	исполняемый Python-код	структурированный вызов (как JSON)
Гибкость	высокая — циклы, условия, комбинирование	ниже — один вызов инструмента за шаг
Предсказуемость	ниже, требует песочницы	выше, проще валидировать
Когда использовать	сложная многошаговая логика	простые, хорошо предсказуемые задачи

Оба типа агентов — частные случаи базового класса `MultiStepAgent`, который реализует уже знакомый цикл Thought-Action-Observation через последовательность шагов: `SystemPromptStep` (формирует системный промпт с описанием доступных инструментов), `TaskStep` (фиксирует задачу пользователя) и `ActionStep` (записывает каждое действие агента и его результат, накапливая историю).

Создание инструментов в `smolagents`

Как и в `Unit 1`, инструмент можно оформить самым простым способом — декоратором `@tool` над обычной функцией с аннотациями типов и `docstring`-ом:

```
from smolagents import tool

@tool
def get_weather(location: str) -> str:
    """Получает погоду для указанного места.

    Args:
        location: название города.
    """
    return f"Погода в {location}: солнечно, 22°C"
```

Для более сложных инструментов, которым нужно собственное состояние или инициализация, используется полноценный класс — наследник `Tool`, с полями `name`, `description`, `inputs`, `output_type` и методом `forward()`.

Готовый набор инструментов (Default Toolbox)

`smolagents` поставляется с набором готовых к использованию инструментов: `PythonInterpreterTool` (безопасный интерпретатор Python для `CodeAgent`), `FinalAnswerTool` (формально завершает работу агента и возвращает финальный ответ), `UserInputTool` (запрашивает дополнительный ввод у пользователя), `DuckDuckGoSearchTool` и `GoogleSearchTool` (веб-поиск), `VisitWebpageTool` (загружает и читает содержимое веб-страницы).

Обмен инструментами и агентами через Hugging Face Hub

Один из плюсов экосистемы Hugging Face — лёгкий обмен готовыми компонентами. Инструмент можно опубликовать вызовом `push_to_hub()` и затем подключить в любом другом проекте через `Tool.from_hub()`. Точно так же можно переиспользовать инструмент, опубликованный как отдельное Space (`Tool.from_space()`), импортировать инструменты из экосистемы LangChain (`Tool.from_langchain()`) или подключить сразу целый набор инструментов с MCP-сервера через `ToolCollection.from_mcp()` — то есть тот самый протокол MCP из `Unit 1` напрямую интегрирован в `smolagents`.

Retrieval и Agentic RAG в `smolagents`

`smolagents` позволяет реализовать поиск по собственным данным (retrieval) как обычный инструмент. Типичный пример — `BM25Retriever` (классический алгоритм текстового поиска без эмбедингов) поверх документов, предварительно разбитых на фрагменты через `RecursiveCharacterTextSplitter`. Более продвинутые сценарии Agentic RAG добавляют к простому поиску переформулировку запроса (query reformulation), разбиение сложного запроса на подзапросы (decomposition), расширение запроса синонимами (expansion) и переранжирование найденных результатов (reranking) — агент сам решает, какие из этих приёмов применить, а не следует жёсткому сценарию.

Мультиагентные системы

Когда задача становится слишком сложной для одного агента, smolagents позволяет собрать иерархию: один `manager_agent` координирует работу нескольких `managed_agents`, каждый из которых специализируется на своей части задачи (например, один агент ищет информацию в интернете, другой — считает, третий — пишет итоговый отчёт). Такую структуру можно визуализировать методом `agent.visualize()`, чтобы наглядно увидеть, кто кому подчиняется и какими инструментами располагает.

Агенты с изображениями и браузером

Для задач, где агенту нужно «видеть», smolagents поддерживает модели VLM (Vision Language Model) — например, через `OpenAIServerModel` с моделью `gpt-4o`, — которым можно передавать изображения параметром `images`. На этой основе построены браузерные агенты: с помощью `Selenium` и обёртки `Helium` агент кликает по реальным веб-страницам, а специальные `step_callbacks` делают скриншот после каждого шага, чтобы у модели была актуальная картинка происходящего на экране.

Наблюдаемость (observability)

Чтобы отлаживать и анализировать поведение агента в продакшене, smolagents поддерживает стандарт `OpenTelemetry`: подключив `SmolagentsInstrumentor` и сервис вроде `Langfuse`, можно получить подробные трассировки (traces) каждого запуска агента — какие мысли, действия и наблюдения происходили на каждом шаге, сколько времени и токенов это заняло.

2.2. Фреймворк LlamaIndex

LlamaIndex изначально создавался как библиотека для построения поисковых систем по собственным данным (RAG), и это по-прежнему его сильная сторона — агенты в LlamaIndex, по сути, надстройка над мощным механизмом поиска и работы с данными.

Вся библиотека держится на четырёх ключевых понятиях. `Components` (компоненты) — это базовые строительные блоки: загрузчики данных, эмбединги, хранилища, парсеры и т.д. `Tools` (инструменты) — обёртки, дающие агенту конкретную возможность, от вызова функции до обращения к целому поисковому движку. `Agents` (агенты) — компоненты, которые автоматически используют инструменты и рассуждают над задачей. `Workflows` (рабочие процессы) — структурированные, управляемые событиями последовательности шагов, которые организуют логику приложения в целом.

Полезная особенность экосистемы — LlamaHub, огромный реестр готовых интеграций (загрузчиков данных, инструментов, векторных хранилищ, LLM-провайдеров), которые устанавливаются по единообразной схеме `pip install llama-index-{тип}-{провайдер}`, например `llama-index-vector-stores-chroma`.

RAG-конвейер: пять этапов

LlamaIndex явно выделяет пять последовательных этапов построения RAG-системы, и понимание этой структуры — ключ ко всему фреймворку.



На этапе Loading данные подгружаются в память — например, через SimpleDirectoryReader, который умеет читать целые папки с файлами разных форматов. На этапе Indexing текст преобразуется в числовые векторы — эмбединги — через конвейер вроде IngestionPipeline, который обычно объединяет SentenceSplitter (разбиение текста на фрагменты) и HuggingFaceEmbedding (получение эмбедингов конкретной моделью). На этапе Storing эти векторы сохраняются в специализированную базу данных, например ChromaDB, обёрнутую в ChromaVectorStore, поверх которой строится VectorStoreIndex. На этапе Querying индекс превращается в интерфейс для вопросов — as_retriever() просто возвращает похожие фрагменты, as_query_engine() дополнительно формирует связный ответ через LLM, а as_chat_engine() поддерживает диалог с памятью о предыдущих репликах. Наконец, этап Evaluation оценивает качество: FaithfulnessEvaluator проверяет, не выдумал ли ответ LLM ничего от себя, AnswerRelevancyEvaluator — насколько ответ релевантен вопросу, а CorrectnessEvaluator — насколько он фактически верен.

Для формирования финального ответа из нескольких найденных фрагментов ResponseSynthesizer предлагает разные стратегии: refine — последовательно уточнять ответ, проходя фрагмент за фрагментом; compact — сначала сжать все фрагменты в минимальное число промптов, а затем обработать; tree_summarize — построить из фрагментов дерево и суммировать снизу вверх. За наблюдаемостью и трассировкой запросов удобно следить через LlamaTrace или Arize Phoenix.

Инструменты в LlamaIndex

В LlamaIndex различают четыре типа инструментов. FunctionTool превращает произвольную Python-функцию в инструмент агента (аналог @tool из smolagents). QueryEngineTool оборачивает уже готовый поисковый движок (QueryEngine) в инструмент — благодаря этому агенты можно строить друг над другом, используя один агент как инструмент для другого. ToolSpecs — это подготовленные сообществом наборы взаимодополняющих инструментов под конкретный сервис, например GmailToolSpec для работы с почтой. Utility Tools решают более техническую задачу — не дать результату внешнего вызова переполнить контекст модели: OnDemandToolLoader объединяет загрузку, индексацию и запрос данных в один вызов «по требованию», а LoadAndSearchToolSpec берёт любой существующий инструмент и превращает его в пару "загрузчик + поиск".

LlamaIndex также поддерживает протокол MCP: инструмент McpToolSpec умеет подключаться напрямую к запущенному MCP-серверу и превращать его возможности в обычные инструменты агента.

Агенты в LlamaIndex

Фреймворк поддерживает три вида рассуждающих агентов: Function Calling Agents (для моделей со встроенной поддержкой вызова функций — быстрее и надёжнее), ReAct Agents (работают с любой

моделью, поддерживающей диалог, и подробно показывают ход рассуждений — по аналогии с ReAct из Unit 1) и продвинутые кастомные агенты для нестандартных сценариев на основе BaseWorkflowAgent.

По умолчанию агенты в LlamaIndex не хранят память между независимыми запусками — но её легко подключить через объект Context, который передаётся из вызова в вызов и позволяет агенту помнить, например, что пользователь представился по имени в предыдущей реплике.

LlamaIndex также напрямую поддерживает мультиагентные системы через класс AgentWorkflow: каждому агенту дают имя и описание, один назначается корневым (root_agent), и агенты могут передавать задачу друг другу (handoff), если понимают, что вопрос находится вне их специализации.

Workflows: управляемые события конвейеры

Workflow в LlamaIndex — это способ организовать код в виде последовательности шагов (Step), которые запускаются событиями (Event) и сами порождают новые события. У каждого workflow обязательно есть StartEvent и StopEvent, а между ними можно строить произвольно сложные цепочки: последовательные шаги, ветвления по условию и даже циклы — оператор `|` в аннотации типа позволяет шагу как принимать LoopEvent на вход, так и возвращать его, создавая тем самым повторяющийся участок работы. Отдельная приятная возможность — `draw_all_possible_flows()`, которая рисует граф workflow в HTML-файл для наглядности. Для передачи данных между шагами, которые не укладываются в обычную цепочку событий, используется объект Context с методами `ctx.store.set()/ctx.store.get()`.

Как более высокоуровневая альтернатива ручному построению workflow, класс AgentWorkflow позволяет собрать мультиагентную систему в несколько строк, передав список агентов и корневого агента, а также, при необходимости, общее начальное состояние (initial_state), которое инструменты агентов могут читать и обновлять через Context — например, вести общий счётчик вызовов функций.

2.3. Фреймворк LangGraph

LangGraph — это фреймворк от команды LangChain, который управляет потоком выполнения приложений с LLM. В отличие от LangChain, который даёт стандартный интерфейс для моделей и инструментов, LangGraph — отдельный пакет, сфокусированный именно на оркестрации: как соединить шаги друг с другом, в каком порядке и с какими условиями.

Когда выбирать LangGraph: контроль против свободы

Проектируя агентное приложение, разработчик всегда балансирует между свободой (модели дают больше пространства для творческих и неожиданных решений) и контролем (гарантируется предсказуемое поведение и соблюдаются установленные ограничения). Код-агенты вроде тех, что в smolagents, находятся ближе к полюсу свободы — они могут за один шаг вызвать несколько инструментов, придумать свои промежуточные шаги и т.д., но за счёт этого менее предсказуемы. LangGraph — на другом конце спектра: он особенно ценен там, где нужен предсказуемый процесс, состоящий из чётко определённых шагов с точками принятия решений, даже если внутри отдельных шагов используется LLM.

LangGraph особенно уместен, когда: процесс требует многошагового рассуждения с явным контролем потока; между шагами нужно сохранять состояние; в системе сочетается детерминированная логика с возможностями ИИ; требуется вмешательство человека в процесс (human-in-the-loop); архитектура агента состоит из нескольких взаимодействующих компонентов.

Базовые строительные блоки

В основе LangGraph лежит направленный граф. State (состояние) — это определяемая пользователем структура данных, которая хранит всю информацию, необходимую для принятия решений, и передаётся между узлами. Nodes (узлы) — обычные Python-функции: каждая принимает состояние на входе, что-то делает (вызывает LLM, использует инструмент, принимает решение) и возвращает обновление состояния. Edges (рёбра) соединяют узлы и определяют возможные переходы между ними — они бывают прямыми (всегда из узла A в узел B) или условными (следующий узел выбирается в зависимости от текущего состояния).

```
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END

class State(TypedDict):
    graph_state: str

def node_1(state):
    return {"graph_state": state['graph_state'] + " I am"}

builder = StateGraph(State)
builder.add_node("node_1", node_1)
builder.add_edge(START, "node_1")
builder.add_edge("node_1", END)
graph = builder.compile()
graph.invoke({"graph_state": "Hi, this is Lance."})
```

Всё это собирается в StateGraph — контейнер, который хранит весь граф целиком: в него добавляются узлы (add_node), рёбра (add_edge) и условные рёбра (add_conditional_edges), а затем граф компилируется (compile()) и становится готовым к выполнению (invoke()) или визуализации в виде диаграммы.

Пример 1: Алфред сортирует почту

Первый практический пример — агент Алфред разбирает входящие письма Мистера Уэйна. Состояние (EmailState) хранит само письмо, категорию, признак спама, черновик ответа и историю сообщений с LLM. Граф состоит из узлов read_email → classify_email → (условная развилка по результату классификации) → либо handle_spam (письмо помечается спамом и отбрасывается), либо draft_response + notify_mr_hugg (для настоящего письма готовится черновик ответа и Мистер Уэйн получает уведомление с этим черновиком на утверждение). Ключевая деталь — условное ребро add_conditional_edges("classify_email", route_email, {...}), которое направляет поток в разные узлы в зависимости от того, вернула ли функция-роутер "spam" или "legitimate". Обратите внимание: раз здесь не происходит вызова инструментов, формально это не агент в полном смысле — скорее управляемый LLM-конвейер, что хорошо иллюстрирует именно механику LangGraph.

Пример 2: анализ документов (настоящий агент с инструментами и циклом)

Второй пример — уже полноценный агент по паттерну ReAct: Алфред анализирует документы Мистера Уэйна (например, фото с расписанием тренировок и рационом), используя инструмент extract_text (модель компьютерного зрения читает текст с изображения) и инструмент divide (для вычислений). Граф здесь состоит всего из двух узлов — assistant (модель с привязанными инструментами) и tools (исполнение вызванных инструментов) — соединённых в цикл: после assistant специальная функция tools_condition проверяет, запросила ли модель вызов инструмента; если да — поток идёт в tools и затем возвращается обратно в assistant; если нет — работа завершается в END. Этот цикл в точности повторяет

уже знакомый Thought-Action-Observation из Unit 1, но выраженный явным графом состояний, а не скрытым циклом внутри библиотеки.

LangGraph также интегрируется с сервисами наблюдаемости вроде Langfuse через CallbackHandler, что даёт полную трассировку выполнения графа — какой узел когда сработал, что вернул, сколько это заняло.

Раздел 3. Практика: Agentic RAG на примере гала-приёма

Третий раздел — это сквозной практический пример, объединяющий всё изученное в Разделе 2. Сценарий такой: пользователь устраивает самый роскошный гала-приём века, а агент Алфред должен взять на себя всю подготовку и обслуживание гостей — отвечать на вопросы о гостях, следить за погодой ради точного запуска фейерверков и поддерживать светскую беседу об известных ИИ-исследователях среди приглашённых. Пример специально даётся сразу для всех трёх фреймворков (smolagents, LlamaIndex, LangGraph), чтобы показать, что одна и та же архитектура агента реализуется по-разному, но по общим принципам.

Инструмент для поиска по гостям (RAG)

Данные о гостях (agents-course/unit3-invitees) хранят имя, степень родства/знакомства с хозяином, короткую биографию и email. Чтобы Алфред мог быстро находить нужного гостя, датасет сначала превращается в список документов, а затем поверх них строится инструмент-ретривер на основе BM25Retriever — классического алгоритма текстового поиска, не требующего предварительного вычисления эмбедингов. У инструмента задаются понятные имя и описание (guest_info_retriever), а его метод forward() возвращает несколько наиболее релевантных документов по свободному текстовому запросу вроде «Расскажи о леди Аде Лавлейс».

Дополнительные инструменты Алфреда

Помимо поиска по гостям, Алфреду дают ещё три инструмента. DuckDuckGoSearchTool даёт доступ к свежим новостям и общей информации об окружающем мире. WeatherInfoTool (в учебных целях — с фиктивными данными) позволяет свериться с прогнозом погоды, чтобы не испортить фейерверк дождём. HubStatsTool обращается к Hugging Face Hub и находит самую скачиваемую модель конкретного автора — это даёт Алфреду тему для разговора с гостями-исследователями об их собственных моделях.

Сборка полного агента

Все четыре инструмента (поиск по гостям, веб-поиск, погода, статистика Hub) передаются одному агенту вместе с моделью — например, CodeAgent(tools=[...], model=model, add_base_tools=True, planning_interval=3) в случае smolagents. Параметр planning_interval заставляет агента периодически останавливаться и переосмысливать общий план, а не только реагировать на последнее наблюдение — это особенно полезно в длинных, многошаговых беседах на протяжении целого приёма.

Показательный сквозной пример — просьба «Мне нужно поговорить с доктором Николой Теслой о последних достижениях в области беспроводной энергии, помоги подготовиться». Чтобы ответить, агенту приходится скомбинировать сразу два инструмента: сначала найти в базе гостей, кто такой Тесла и чем он увлекается, затем поискать в интернете последние новости о беспроводной передаче энергии — и только после этого собрать связный ответ с конкретными темами для разговора.

Память между репликами

Интересное наблюдение курса: ни один из трёх фреймворков не встраивает память в агента «по умолчанию» — и это осознанное архитектурное решение, а не недосмотр. В smolagents память не сохраняется между отдельными вызовами run(), если явно не передать reset=False. В LlamaIndex для памяти нужно явно завести и передавать объект Context. В LangGraph для этого предусмотрен либо

ручной доступ к истории сообщений, либо готовый компонент MemorySaver. Такое разделение даёт разработчику осознанный контроль: включать память только тогда, когда сценарий действительно этого требует, вместо того чтобы автоматически копировать контекст на каждый вызов.

Раздел 4. Финальный проект: создать, протестировать и сертифицировать своего агента

Четвёртый, заключительный раздел курса — полностью практический. Здесь нужно самостоятельно собрать своего агента и прогнать его через реальный бенчмарк, чтобы получить второй, более весомый сертификат — «Certificate of Completion». Курс прямо предупреждает: этот раздел требует более продвинутых навыков программирования и предполагает намного меньше пошаговых подсказок, чем предыдущие три.

Бенчмарк GAIA

GAIA — это бенчмарк для оценки ИИ-ассистентов на задачах, близких к реальным: он требует одновременно рассуждения, работы с разными модальностями (текст, изображения), веб-серфинга и грамотного использования инструментов. Бенчмарк включает 466 тщательно отобранных вопросов, которые концептуально просты для человека, но на удивление сложны для современных ИИ-систем — это разрыв специально заложен в дизайн бенчмарка. Для сравнения: люди решают около 92% вопросов, GPT-4 с плагинами — около 15%, а система Deep Research от OpenAI показала 67,36% на валидационной выборке.

GAIA построен на нескольких принципах: сложность как в реальном мире (задачи требуют многошагового рассуждения, мультимодальности и использования инструментов), интерпретируемость для человека (при всей сложности для ИИ, задачи остаются понятными и проверяемыми людьми), невозможность «сжульничать» (правильный ответ требует реального выполнения задачи, а не удачного угадывания) и простота проверки (ответы короткие, фактические и однозначные — что удобно для автоматической оценки).

Вопросы разбиты на три уровня сложности: Level 1 — меньше пяти шагов и минимальное использование инструментов; Level 2 — более сложное рассуждение и координация нескольких инструментов, 5–10 шагов; Level 3 — долгосрочное планирование и продвинутая интеграция разных инструментов. Пример вопроса уровня Level 3 из курса показывает масштаб сложности: «Какие из фруктов, изображённых на картине 2008 года 'Embroidery from Uzbekistan', подавались на завтрак в октябре 1949 года на океанском лайнере, который позже использовался как плавучий реквизит в фильме 'The Last Voyage'? Перечислите их по часовой стрелке, начиная с позиции на 12 часов» — чтобы ответить, агенту нужно распознать фрукты на изображении, определить нужный лайнер, найти архивное меню и правильно упорядочить результат.

Практическое задание и API

Для этого раздела курс предоставляет отдельный API с четырьмя основными точками входа: GET /questions возвращает полный список из 20 отобранных вопросов уровня Level 1 (специально подобранных по числу нужных шагов и инструментов, чтобы 30% результата были достижимой целью); GET /random-question отдаёт один случайный вопрос; GET /files/{task_id} скачивает файл, приложенный к конкретному вопросу (например, изображение или таблицу); POST /submit принимает ответы агента, сверяет их с эталоном строго по точному совпадению (exact match) и обновляет учебный лидерборд.

Именно поэтому важно правильно "промптить" агента: раз ответ сверяется дословно, агент должен возвращать только сам ответ, без пояснений и без служебных фраз вроде "FINAL ANSWER". Курс даёт студентам стартовый шаблон агента, который дублируется на свой аккаунт Hugging Face как отдельное

Space, после чего его можно свободно менять и дополнять любыми инструментами. Для самой отправки результата нужны три вещи: имя пользователя Hugging Face (для идентификации), публичная ссылка на код агента (Space должен оставаться открытым для проверки) и, собственно, список ответов агента на вопросы.

Важно понимать: чтобы реально получить 30%+ и второй сертификат, нужно создать собственное Space на Hugging Face Hub под своим аккаунтом, написать и протестировать код агента, прогнать его через вопросы API и отправить результат на публичный лидерборд. Это принципиально другой по масштабу шаг по сравнению с чтением материала: он требует активных действий в собственном аккаунте пользователя, поэтому решение о том, стоит ли за это браться — и в каком объёме — остаётся за вами.

Получение сертификата «Certificate of Completion»

Если результат на бенчмарке превысил 30%, сертификат получают так же, как и в Unit 1: заходят на отдельную страницу сертификата, авторизуются через Hugging Face, вводят полное имя, которое должно появиться на сертификате, и нажимают «Get My Certificate» — система проверяет результат на лидерборде и выдаёт сертификат.

Дальнейшее развитие

Курс на этом формально завершается, но предлагает продолжение для желающих: три бонусных модуля — тонкая настройка (fine-tuning) LLM под вызов функций, наблюдаемость и оценка агентов (Bonus Unit 2) и агенты, играющие в игры на примере Pokémon (Bonus Unit 3), — а также общие рекомендации по дальнейшему изучению темы, включая более глубокое знакомство с протоколом MCP.



Бонусные модули

Помимо основных четырёх разделов, курс предлагает три необязательных бонусных модуля — каждый расширяет тему в свою сторону: дообучение моделей под вызов функций, наблюдаемость и оценка агентов в продакшене, и агенты в видеоиграх. Формальной сертификации за них не предусмотрено, но они хорошо дополняют общую картину.

Бонусный модуль 1. Дообучение LLM для вызова функций (Function-Calling)

Этот модуль возвращается к теме инструментов, но заходит с другой стороны. В Разделе 1 агент получал список инструментов через промпт и просто обобщал, как ими пользоваться — сама модель этому специально не обучалась. Function-calling — противоположный подход: способность вызывать инструменты закладывается в модель через дообучение (fine-tuning), а не только через промпт-инжиниринг.

Технически это выражается в новых ролях внутри диалога и новых служебных токенах. Помимо привычных user/assistant сообщений появляются вызов инструмента (tool call) и результат его работы (tool result). Например, у моделей семейства Mistral это оформляется как отдельное поле `function_call` в ассистентском сообщении и отдельная роль `tool` для результата, а на уровне токенизации — специальными токенами вроде `[AVAILABLE_TOOLS]`, `[TOOL_CALLS]`, `[TOOL_RESULTS]`. По сути, это более строгая, «зашитая в модель» версия того же цикла Thought-Action-Observation из Раздела 1.



Чтобы добавить такую способность модели, которая её изначально не умеет (в качестве примера курс берёт google/gemma-2-2b-it), нужно понимать три стадии, через которые проходит обучение LLM. Предобучение (pre-training) на огромном корпусе текста даёт базовую модель, которая только предсказывает продолжение текста и не умеет вести диалог. Инструктивное дообучение (instruction tuning) учит её следовать инструкциям и поддерживать диалог — так появляются модели с суффиксом -it/-Instruct. Alignment донастраивает модель под конкретные ценности и стиль поведения, заданные её создателем (например, вежливость в диалогах поддержки).

Курс намеренно начинает не с базовой модели, а сразу с инструктивной (gemma-2-2b-it): так модели нужно освоить только один новый навык — вызов функций, — а не сразу учиться следовать инструкциям и вызывать функции одновременно. Для самого дообучения используется LoRA (Low-Rank Adaptation) — облегчённая техника, которая не переобучает все веса модели заново, а добавляет поверх неё сравнительно небольшой набор новых обучаемых весов (адаптер). Это заметно снижает требования к

вычислительным ресурсам, а результат — компактные веса адаптера размером в сотни мегабайт, которые легко хранить и распространять отдельно от основной модели, без потери скорости на этапе инференса после объединения адаптера с моделью.

Бонусный модуль 2. Наблюдаемость и оценка агентов

Этот модуль адресован ситуации, с которой рано или поздно сталкивается любой, кто выводит агента «в люди»: без специальных средств агент — чёрный ящик. Наблюдаемость (observability) — это работа с внешними сигналами о поведении системы: логами, метриками и трассировками (traces) — которые превращают агента из чёрного ящика в прозрачную, отлаживаемую систему, где видно стоимость запросов, точность ответов, задержки, потенциальные риски безопасности (например, попытки prompt injection) и обратную связь пользователей.

Ключевое понятие здесь — пара «трасса и спан» (traces and spans). Трасса (trace) — это весь путь одного запроса агента от начала до конца, например обработка одного вопроса пользователя целиком. Спан (span) — это отдельный шаг внутри трассы: один вызов модели, один вызов инструмента, одно обращение к базе данных. Многие фреймворки, включая smolagents, используют для этого открытый стандарт OpenTelemetry, что позволяет подключать разные инструменты наблюдаемости (например, Langfuse или Arize) без привязки к конкретному фреймворку.

Метрика	Что показывает	Как реагировать
Задержка (latency)	сколько времени занимает ответ агента и отдельные шаги	ускорить модель, распараллелить вызовы
Стоимость (costs)	расходы на токены и внешние API за один прогон	сократить число вызовов, выбрать модель дешевле
Ошибки запросов	сколько вызовов модели/инструментов завершились сбоем	добавить retry и запасные (fallback) варианты
Обратная связь пользователей	явные оценки (👍/👎, звёзды) и косвенные сигналы (переспрашивание)	сигнал о реальном качестве в проде
Точность (accuracy)	доля правильных/желаемых ответов по заданному критерию успеха	нужен чёткий критерий именно для вашей задачи
Автоматическая оценка	LLM-судья или библиотеки вроде RAGAS/LLM Guard	программная, воспроизводимая оценка без участия человека

Модуль разделяет оценку агентов на два взаимодополняющих подхода. Оффлайн-оценка (offline evaluation) проводится в контролируемых условиях — на заранее подготовленном наборе тестовых вопросов с известными правильными ответами (например, GSM8K — набор математических задач); это позволяет получить воспроизводимую, чёткую метрику точности и удобно для CI/CD, но требует поддерживать тестовый набор актуальным. Онлайн-оценка (online evaluation) отслеживает поведение агента уже в проде, на реальных запросах живых пользователей — она ловит то, что невозможно предугадать в лаборатории (дрейф модели, неожиданные формулировки запросов), но полагается на менее надёжные сигналы вроде пользовательской обратной связи. На практике лучшие результаты даёт связка обоих подходов в постоянном цикле: офлайн-тесты → выкладка новой версии агента → мониторинг в проде → новые проблемные примеры добавляются обратно в офлайн-набор → и по кругу.

Практическая часть модуля показывает, как подключить Langfuse к агенту на smolagents, обогатить трассировки метаданными (ID пользователя, ID сессии), встроить сбор обратной связи прямо в чат-интерфейс на Gradio и настроить отдельную LLM-модель в роли «судьи», которая оценивает ответы агента на токсичность или точность.

Бонусный модуль 3. Агенты в играх (на примере Pokémon)

Заключительный бонусный модуль переносит идею агентов в игровую индустрию — на примере NPC (неигровых персонажей). Модуль проводит чёткую границу между двумя уровнями «умности» персонажа. NPC на основе LLM даёт живые, небанальные реплики, но остаётся статичным и реактивным — он отвечает, только когда к нему обращается игрок. Агентный NPC (Agentic AI NPC) способен на самостоятельные решения — «отправиться за подмогой, расставить ловушку или вовсе избегать игрока» — без прямого запроса со стороны игрока.

Такое поведение опирается на три свойства: автономность (самостоятельные решения на основе состояния игры), адаптивность (стратегическая перестройка поведения в ответ на действия игрока) и постоянство (память о прошлых взаимодействиях, влияющая на будущее поведение). Модуль честно называет и главное ограничение: задержка. Рассуждающему агенту нужно достаточно токенов, чтобы «подумать» и сформировать действие, а это плохо совместимо с играми реального времени (стандарт — 30 кадров в секунду); в качестве примера курс упоминает эксперимент "Claude Plays Pokémon", иллюстрирующий именно эту проблему. Поэтому на практике агентный подход сегодня лучше всего подходит именно к пошаговым играм (turn-based), где у агента объективно есть время подумать между ходами — Pokémon-баттлы тому яркий пример.

Курс приводит несколько показательных примеров того, куда уже пришли LLM в играх: демо Covert Protocol (NVIDIA + Inworld AI) с NPC-детективами, реагирующими на реплики игрока в реальном времени; прототип NEO NPCs от Ubisoft — персонажи, способные воспринимать окружение, помнить прошлые разговоры и вести содержательный диалог; демо Mecha BREAK с NPC на технологии NVIDIA ACE, распознающими игрока и предметы через веб-камеру благодаря GPT-4o; и уже вышедшая игра Suck Up! — там нужно уговорить NPC-хозяев впустить вас в дом, причём каждый персонаж управляется генеративным ИИ и реагирует непредсказуемо.

Практика: свой боевой агент для Pokémon

Хендс-он часть модуля — сборка собственного агента для пошаговых боёв в стиле Pokémon, из четырёх компонентов. Poke-env — Python-библиотека (изначально для обучения RL-ботов), которая даёт агенту простой API для взаимодействия с симулятором боёв. Pokémon Showdown — открытый онлайн-симулятор, где агент реально сражается — сервер для этого предоставляет сам курс, чтобы все участники бились в одном месте. LLMAgentBase — заготовленный курсом класс на основе Poke-env's Player, который берёт на себя всю рутину: превращает состояние боя в текстовое описание для LLM (какой покемон активен, его HP, доступные ходы и их сила/точность, доступные замены, погода на поле и т.д.), а затем разбирает ответ модели и превращает его в реальный игровой ход через два инструмента — choose_move (выбрать атаку) и choose_switch (сменить покемона). TemplateAgent — стартовый шаблон, который нужно доделать: реализовать единственный абстрактный метод _get_llm_decision(battle_state), то есть решить, как именно запрашивать свою LLM и разбирать её ответ в один из двух вызовов инструментов.

```
class LLMAgentBase(Player):
    def _format_battle_state(self, battle) -> str:
```



```

# активный покемон, HP, статус, доступные ходы и замены -> в текст для LLM
...

async def choose_move(self, battle) -> str:
    state_str = self._format_battle_state(battle)
    decision = await self._get_llm_decision(state_str) # реализуется в наследнике
    # разбор decision["name"] == "choose_move" / "choose_switch" -> реальный ход
    # при ошибке LLM - запасной случайный ход, чтобы бой не завис
    ...

async def _get_llm_decision(self, battle_state: str):
    raise NotImplementedError # здесь свой запрос к LLM и разбор ответа

```

Курс даёт готовые примеры реализации TemplateAgent для OpenAI, Mistral и Gemini — как ориентир, а не готовое решение «из коробки». Идея финального шага модуля — задеплоить своего агента и сразиться с чужими агентами в реальном времени на общем сервере, что превращает обучающее упражнение в живое соревнование.

Общий итог по курсу

К этому моменту весь материал курса «AI Agents Course» законспектирован целиком: основы устройства агентов и LLM (Раздел 1), три конкретных фреймворка для их реализации — smolagents, LlamaIndex и LangGraph (Раздел 2), сквозной практический пример Agentic RAG (Раздел 3), структура финального проекта на бенчмарке GAIA (Раздел 4), и все три бонусных модуля — дообучение под вызов функций, наблюдаемость и оценка агентов, агенты в играх.

Первый сертификат — «Certificate of Fundamentals of Agents» — уже получен по результатам теста с итогом 90%. Второй, более весомый сертификат — «Certificate of Completion» — требует самостоятельной публикации собственного агента на Hugging Face Hub и прохождения им не менее 30% вопросов бенчмарка GAIA; готовый код для этого агента подготовлен отдельно (см. переданный архив с проектом), а сама публикация — по решению читателя, отдельным шагом, в своём аккаунте.